

13. How do Servlets and JSPs work together in a web application following the MVC (Model-View-Controller) architecture? Provide a brief use case showing the servlet as a controller, JSP as a view, and Java class as the model.

MVC Roles -

- Model → Java class (business logic)
- View → JSP (presentation)
- Controller → Servlet (request handling)

Use case: Display Student Name

Model (Student.java)

```
public class Student {  
    public String getName () {  
        return "Ali";  
    }  
}
```

Controller (StudentServlet.java)

@WebServlet ("*/student")

```
public class StudentServlet extends HttpServlet {  
    protected void doGet (HttpServletRequest request,  
                           HttpServletResponse response)  
        throws ServletException, IOException {  
        Student student = new Student ();  
        request.setAttribute ("name", student.getName ());  
        RequestDispatcher rd = request.getRequestDispatcher ("view.jsp");  
        rd.forward (request, response);  
    }  
}
```

View (view.jsp)

<h2> Student Name : {name} </h2>

Servlet processes request → calls method → forwards data to JSP → JSP displays result.

19. Explain the lifecycle of a `Servlet`. What are the roles of the `init()`, `service()` and `destroy()` methods? Discuss how servlets handle concurrent requests and how thread safety issues may arise.

1. `init()`

- Called once when servlet is created.
- Used for initialization

2. `Service()`

- Called for each request
- Handles client requests
- Internally calls `doGet()` or `doPost()`

3. `destroy()`

- Called once before servlet removal.
- Used to release resources.

The lifecycle of a servlet is managed by the `Servlet Container` (e.g. `Tomcat`). It consists the following five stages:-

- i) Loading
- ii) Instantiation
- iii) Initialization (`init()`)
- iv) Request Handling (`service()`)
- v) Destruction (`destroy()`).

Handling Concurrent Requests:

- Only one servlet instance is created
- Multiple threads handle multiple requests.
- Threads share instance variables.

Thread Safety Issue:

If instance variables are shared, race conditions may occur.

15. A single instance of a servlet handles multiple requests using threads. What problems can occur if shared resources (e.g. variables) are accessed by multiple threads. Illustrate your answer with an example and suggest a solution using synchronization.

If multiple threads access shared variables, inconsistent results may occur.

Example:

```
public class CounterServlet extends HttpServlet {  
    int count = 0;  
    protected void doGet (HttpServletRequest req, Http  
        HttpServletResponse res)  
        throws IOException {  
        count++;  
        res.getWriter().println ("Count: " + count);  
    }  
}
```

Problem:

Multiple threads increment count.
Race condition may occur.

Solution (synchronization):

```
protected synchronized doGet void doGet (HttpServletRequest req,  
    HttpServletResponse res)  
    throws IOException {  
    count++;  
    res.getWriter().println ("Count: " + count);  
}
```

Or local variables can be used instead of instance variables.

16. Describe how the MVC (Model-View-Controller) pattern separates concerns in a Java Web Application. Explain the advantages of this structure in terms of maintainability and scalability, using a student registration system as an example.

MVC Overview

MVC (Model-View-Controller) separates an application into three components:

- Model - Handles business logic and data (Java classes, database operations)
- View - Displays data to the user (JSP/HTML)
- Controller - Handles user requests and controls application flow (Servlet)

Example: Student Registration System

- Model - Student.java (stores student data, interacts with database)
- View - register.jsp (registration form)
- Controller - StudentServlet (processes form data)

Flow:

1. User submits registration form.
2. Servlet receives request.
3. Servlet calls Model to save student data.
4. Servlet forwards result to JSP for display.

Advantages

1. Maintainability

- Business logic separated from UI.
- Changes in UI do not affect backend logic.
- Easier debugging and testing.

2. Scalability

- Each component can be extended independently.
- Multiple developers can work on different layers.
- Suitable for large applications.

MVC improves organization, code clarity, maintainability, and scalability in Java web applications.

18. Compare and contrast cookies, URL rewriting, and HttpSession as methods for session tracking in servlets. Discuss their advantages, limitations and ideal use cases.

1. Cookies

- Stored on client browser.
- Sent automatically with each request.

Advantages:

- Simple to use.
- Lightweight.

Limitations:

- Can be disabled by user.
- Limited storage.
- Security risks if not protected.

Use case:

- Appends session ID to URL.
- Remembering user performance.

2. URL Rewriting

- Appends session ID to URL.

Example:

example.com/page? sessionId = 123

Advantages:

- Works if cookies are disabled.

Limitations:

- Exposes session ID in URL
- Less secure
- Messy URLs

Use case:

- Small applications without cookies support.

3. Http Session:

- Server-side session management
- Session ID stored in cookie.

Advantages:

- Secure
- Stores complex objects
- Managed by container

Limitations:

- Uses server memory.
- Requires session management

Use case:

- Logging systems, shopping carts.
Login

19. A web application stores user login information using HttpSession. Explain how the session works across multiple requests and how session timeout or invalidation is handled securely.

The session works in the following way:-

1. User logs in.
2. Server creates HttpSession.
3. Session ID is sent to browser (usually via cookie).
4. Browser sends session ID with every request.
5. Server retrieves session using session ID.

Example.

```
HttpSession session = request.getSession();  
session.setAttribute("username", "Ali");
```

Later:

```
String user = (String) session.getAttribute("username");
```

Session Timeout & Invalidation :-

- Default timeout configured in web.xml or server settings.
- Manual invalidation.

```
session.invalidate();
```

Security Practices :

- Use HTTPS.
- Invalidate session on logout.
- Regenerate session ID after login.

20. Explain how Spring MVC handles an HTTP request from browser. Describe the role of the @Controller, @RequestMapping and Model objects in separating business logic from presentation. Provide a brief flow example of login form submission.

Flow of HTTP Request in Spring MVC

1. Browser sends request.
2. DispatcherServlet receives it.
3. Dispatcher maps request to controller.
4. Controller processes request.
5. Returns view name and model data.
6. View renders response.

Key components

@Controller

- Marks class as controller.

@RequestMapping

- Maps URL to specific method.

Model

- Transfers data from controller to view.

Login Form Example

Controller :

@ Controller

```
public class LoginController {
```

```
    @RequestMapping ("/login")
```

```
    public String login (String username, String password, Model model) {
```

```
        if (username.equals ("admin")) {
```

```
            model.addAttribute ("message", "Login Successful");
```

```
            return "success";
```

```
        } else {
```

```
            model.addAttribute ("message", "Login Failed");
```

```
            return login;
```

```
    }
```

Flow :

1. User submits login form.
2. Controller receives data.
3. Business logic executed.
4. Model stores message.
5. View displays result.

Spring MVC separates business logic from presentation using controllers and models, improving maintainability, testability and scalability.

Q1. Spring MVC uses the DispatcherServlet as a front controller. Describe its role in the request processing workflow. How does it interact with view resolvers and handler mapping?

DispatcherServlet acts as the front controller in Spring MVC. It handles all incoming HTTP requests and coordinates processing.

Request Workflow:

1. Client sends HTTP request.
2. DispatcherServlet receives request.
3. It consults HandlerMapping to find appropriate controller.
4. Controller processes request and returns Model + View name.
5. DispatcherServlet sends view name to ViewResolver.
6. ViewResolver resolves actual view (JSP/HTML).
7. Response is returned to client.

Interaction:

- HandlerMapping → Maps URL to controller method.
- ViewResolver → Maps logical view name to actual view.
- DispatcherServlet coordinates both.

24. How does JPA manage the mapping between Java objects and relational tables? Explain with an example using @Entity, @Id and @GeneratedValue annotations. Discuss the advantage of using JPA over raw JDBC.

How JPA manages mapping:

JPA maps Java objects to database tables using annotations.

Example:

```
import jakarta.persistence.*;
```

```
@Entity
```

```
public class Student {
```

```
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;
```

```
    private String name;
```

@Entity → Marks class as database table.

@Id → Primary key

@GeneratedValue → Auto-generates ID.

Advantage over JDBC:

- No manual SQL writing.
- Automatic object-table mapping.
- Reduced boilerplate code.
- Easier maintenance.
- Database independent queries (JPQL).

25. Describe the difference between the EntityManager's persist(), merge() and remove() operations. When would you use each method in a typical database transaction scenario?

1. persist()

- Inserts new entity into database.
- Used for new objects.

Ex: entityManager.persist(student);

2. merge()

- Updates existing entity.
- Used when modifying detached objects.

Ex: entityManager.merge(student);

3. remove()

- Deletes entity from database.

Ex: entityManager.remove(student);

Typical Scenario for each method:

- New record → persist()
- Update record → merge()
- ~~Remove~~ Delete record → remove()

Q8. Compare @RestController and @Controller in Spring Boot
In a RESTful API for a library system, describe how you would structure the endpoints for books using REST principles (e.g., /books, /books/{id}).

@Controller:

- Returns view (JSP/HTML).
- Used in MVC applications.

@RestController:

- Returns JSON/XML data.
- Combines @Controller + @ResponseBody.
- Used in REST APIs.

REST API Structure (Library System)

<u>Method</u>	<u>Endpoint</u>	<u>Description</u>
GET	/books	Get all books
GET	/books/{id}	Get book by ID
POST	/books	Add new book
PUT	/books/{id}	Update book
DELETE	/books/{id}	Delete book

Example:

@RestController

@RequestMapping("/books")

```
public class BookController {
```

@GetMapping

```
    public List<Book> getAllBooks() {}
```

@GetMapping("/{id}")

```
    public Book getBook(@PathVariable Long id) {}
```

@PostMapping

```
    public Book addBook(@RequestBody Book book) {}
```

```
}
```

Q9. How does Maven manage dependencies and build lifecycle in a Spring Boot project? Explain the structure of a typical pom.xml and how the Spring Boot starter dependencies simplify development.

Dependency Management

Maven uses pom.xml to :

- Manage dependencies
- Define plugins
- Control build lifecycle.

Example: (Typical pom.xml structure)

```
<project>
  <parent>
    <groupId> org. springframework. boot </groupId>
    <artifactId> spring-boot-starter-parent </artifactId>
  </parent>
  <dependencies>
    <dependency>
      <groupId> org. springframework. boot </groupId>
      <artifactId> spring-boot-starter-webparent </artifactId>
    </dependency>
  </dependencies>
</project>
```

Spring Boot Starters

- Provide pre-configured dependencies.
- Reduce manual configuration
- Example: spring-boot-starter-web

30. Compare Maven and Gradle as build tools in Spring Boot projects. Discuss their Syntax, performance and dependency handling with an example of a build.gradle configuration for a simple REST API.

Maven vs Gradle :

<u>Feature</u>	<u>Maven</u>	<u>Gradle</u>
Syntax	XML	Groovy/Kotlin DSL
Performance	Slower	Faster (Incremental builds)
Flexibility	Less flexible	More flexible
Dependency Handling	pom.xml	build.gradle

Example : (build.gradle)

```
plugins {  
    id 'org.springframework.boot' version '3.0.0'  
    id 'java'  
}  
  
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-web'  
    testImplementation 'org.springframework.boot:spring-boot-starter-test'  
}
```

31. You are building a multi-module Spring Boot application (e.g., with separate modules for API, service and database layers). Explain how you would structure the project using Maven or Gradle and manage inter-module dependencies effectively.

Structure (Maven Example)

parent-project

```
| api-module  
| service-module  
| repository-module
```

Parent pom.xml

```
<packaging> pom </packaging>
```

```
<modules>
```

```
  <module> api-module </module>
```

```
  <module> service-module </module>
```

```
</modules>
```

Inter-Module Dependency

In api-module :

```
<dependency>
```

```
  <groupId> com.example </groupId>
```

```
  <artifactId> service-module </artifactId>
```

```
</dependency>
```

Benefits :

- Separation of concerns.
- Independent development.
- Better scalability.

33. Socket Programming Example:

Server:

```
import java.net.*;  
import java.io.*;  
  
public class Server {  
    public static void main(String[] args) throws Exception {  
        ServerSocket server = new ServerSocket(5000);  
        ServerSocket = server.accept();  
        DataInputStream in = new DataInputStream(socket.getInputStream());  
        System.out.println(in.readUTF());  
        server.close();  
    }  
}
```

Client:

```
import java.net.*;  
import java.io.*;  
  
public class Client {  
    public static void main(String[] args) throws Exception {  
        Socket socket = new Socket("localhost", 5000);  
        DataOutputStream out = new DataOutputStream(socket.getOutputStream());  
        out.writeUTF("Hello Server");  
        socket.close();  
    }  
}
```

34. RMI Example - Chat system with FX

Remote Interface

```
import java.rmi.*;
```

```
public interface ChatService extends Remote {  
    void sendMessage (String msg) throws Exception;  
}
```

Server Implementation

```
import java.rmi.server.UnicastRemoteObject;
```

```
public class ChatServiceImpl  
    extends UnicastRemoteObject  
    implements ChatService {
```

```
    protected ChatServiceImpl() throws Exception {}
```

```
    public void sendMessage (String msg) {  
        System.out.println ("Client: " + msg);  
    }  
}
```

JavaFX Client (Simplified)

```
ChatService service =
```

```
(ChatService) Naming.lookup ("rmi://localhost/ChatService");
```

```
service.sendMessage ("Hello from JavaFX client");
```

Q2. Describe how JDBC manages communications between a Java application and a relational database. Outline the steps involved in executing a SELECT query and fetching results. Include error handling with try-catch and finally blocks.

JDBC (Java Database Connectivity) allows Java applications to communicate with relational databases using SQL.

Steps to Execute SELECT Query:

1. Load JDBC driver.
2. Establish connection
3. Create statement.
4. Execute query.
5. Process ResultSet.
6. Close resources.

Example:

```
import java.util.
import java.sql.*;

public class JDBCExample {
    public static void main (String[] args) {
        Connection conn = null;
        Statement stmt = null;
        ResultSet rs = null;

        try {
            conn = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/testdb",
                "root",
                "pookie" // password
            );
        }
    }
}
```

```
stmt = conn. create statement();  
rs = stmt. executeQuery ("SELECT * FROM students");  
while (rs.next()) {  
    System.out.println (rs.getInt("id") + " " + rs.getString("name"));  
}
```

```
} catch (SQLException e) {  
    e.printStackTrace();  
}  
} finally {
```

```
try {
```

```
    if (rs != null) rs.close();
```

```
    if (stmt != null) stmt.close();
```

```
    if (conn != null) conn.close();
```

```
} catch (SQLException e) {
```

```
    e.printStackTrace();
```

```
}
```

```
}
```